MODULE 05 / TEAMMATE FIELD GUIDE

Skill Matching + Career GPS

Compare the saved candidate's reviewed named claims with one reviewed JD. Explain matched, missing and unresolved requirements, then let the user explore reversible hypothetical improvements.

JD | Resume | Profile | Talent Check | **Match + GPS**

The shared journey; each module also runs independently.

You receive	You return
Saved profile ID + saved JD ID. GPS also uses their current revisions, optional company/benchmark snapshot, named-skill overrides and category overrides.	MatchingResult with score/null, matched/missing/ambiguous decisions and contributions. ScenarioResult adds baseline/projected outcomes and explicitly hypothetical changes.

Your responsibility

Participant Brief p.8 requires one-specific-JD score plus matched and missing skills. It permits exact/fuzzy matching; an LLM matcher is not required. Importance, ProofPath reasoning and Career GPS are our additions.

A successful handoff looks like this

Different candidates/jobs produce understandable differences. A missing Docker claim has a source-backed reason, a reversible projection, and no silent change to the stored profile.

Start from the working baseline

Assume YOU own this module. Run and understand the supplied working source, finish or improve your part, and test it. Push the verified working copy to YOUR OWN GitHub repository. Return its URL, commit SHA, actual test results and known limitations to the Scrum Master. No central merge or PR is required from you.

Before your module	After your module
Module 1 supplies the reviewed JD; Module 3 supplies the saved profile. Module 4 supplies the readiness evaluator and optional benchmark snapshot for GPS.	The candidate receives a source-to-claim-to-points-to-action explanation. GPS returns a preview only; it never saves capability changes to the real profile.

Read in order: 2 build plan | 3 screen layout | 4 API | 5 logic | 6 integration | 7 tests | 8 presentation | 9 exact field reference

02 / IMPLEMENTATION PLAN

What to do, in order.

Keep changes inside your feature folder. Each step has an observable result; do not begin by replacing the whole app.

01 Load the exact pair

Use saved profile_id and jd_id; display the candidate and job clearly. Company benchmarks and numeric ratings are not needed for named-skill matching.

02 Normalize conservatively

Match exact names and reviewed aliases. Java is not JavaScript. A shared category is not enough to satisfy a named technology.

03 Score requirement groups

Use mandatory weight 2, preferred/unspecified weight 1. AND uses a matched fraction; OR needs any member. Deduplicate equivalent groups.

04 Explain every decision

Preserve JD quote, candidate quote or honest absence, importance, reason and contribution points. Treat fuzzy suggestions and unresolved claims as review tasks, not automatic matches.

05 Record current revisions

Return a result/receipt linked to input revisions and rules. Empty JD is unscorable; unreviewed/ambiguous coverage remains provisional.

06 Preview on a copy

GPS deep-copies the profile, applies only allowed explicit overrides and calls the same match/readiness engines. Hold revisions and benchmark hash fixed.

07 Reset and prove no writes

Reset removes hypothetical changes. Compare saved profile JSON before/after. Do not expose scenario values as actual evidence or save them behind the scenes.

Files to open first

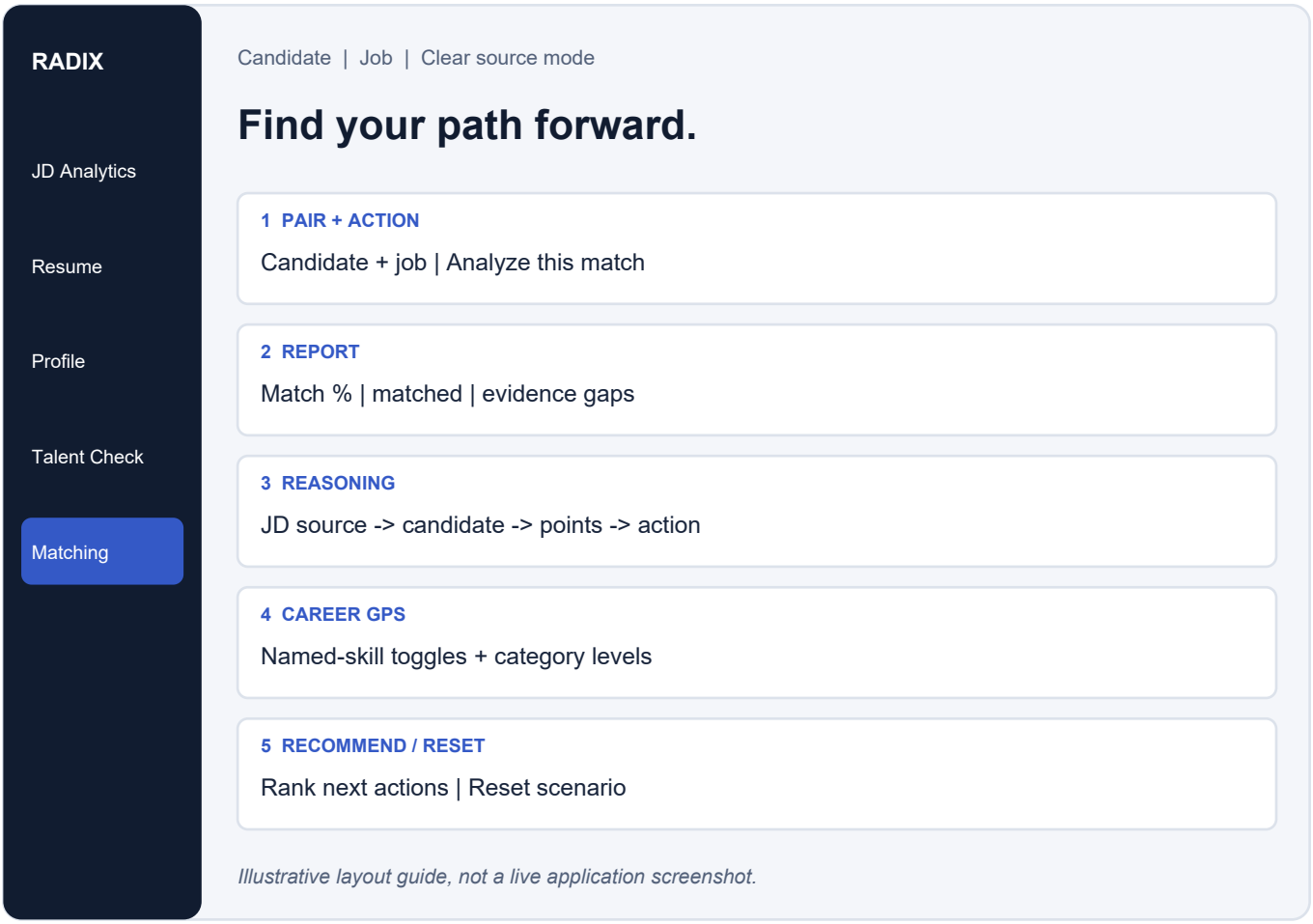
All paths below are relative to skill_matching/.

File	What it owns
backend/matcher.py	Deterministic alias/group coverage and explanations.
backend/scenarios.py	Pure copied-profile projections and same-engine recommendations.
backend/router.py	Loads saved inputs, evaluates and records receipts.
frontend/SkillMatchingPage.tsx	Match report, breakdown, GPS controls and reset.
frontend/ReasoningChain.tsx + MatchLauncher.tsx	Evidence chain and direct CV-to-JD entry flow.

03 / SCREEN AND USER EXPERIENCE

What it should look like.

Keep the current app shell, navigation, typography and shared controls. The drawing shows information order, not a redesign request.



Screen state	Expected visible behavior
Missing candidate or JD	Guide to saved inputs; do not invent an assessment.
Unreviewed/ambiguous input	Label provisional, show unresolved reasons and review route.
Empty JD skills	Unscorable, score null; not 0% or 100%.
Stale scenario revision/hash	409 asks for a fresh baseline; never mix snapshots.

Visual rules: 16 px body/form text, 14 px minimum supporting labels; clear headings; 44 px touch targets. Navy #111C30, blue #3459C6, white cards and #F4F6FA canvas. Use the existing Inter web font. On mobile, stack panels and keep actions readable. Do not rely on color alone for status.

04 / EXACT API BOUNDARY

Input, output and format.

Use the current Pydantic/OpenAPI contract. A UI label can change; an agreed JSON field cannot be renamed independently.

Endpoint	Body or result
POST /api/matching/evaluate	profile_id + jd_id; server loads saved records.
POST /api/matching/scenarios/preview	Pair, current revisions, overrides and optional benchmark snapshot.
GET /api/receipts/ID	Owned stored decision snapshot for the baseline assessment.

Complete baseline evaluation request; use current saved IDs

```
{
  "profile_id": "CURRENT_PROFILE_ID",
  "jd_id": "CURRENT_JD_ID"
}
```

Current synthetic response excerpt

Only selected fields are shown below. Page 9 describes the complete shape. Snapshot UUIDs are teaching examples, not IDs to reuse in a running app.

```
{
  "jd_source_file": "junior-data-engineer-jd-SYNTHETIC.docx",
  "jd_revision": 2,
  "profile_revision": 1,
  "match_score": 75.0,
  "status": "complete",
  "rules_version": "match-1.3-incomplete-extraction-disclosure"
}
```

Every mutation needs X-ProofPath-Client: web (or cli). Cloud requests also need the current user's Bearer JWT. The shared web helper supplies these headers. File upload uses multipart; never manually set its boundary. JSON fields use snake_case.

05 / REASONING AND BUSINESS RULES

Match requirements first. Simulate improvements separately.

A score is weighted coverage of reviewed extracted requirements. It is not measured proficiency. Confidence is not a score multiplier, and unrelated extra skills do not reduce the match.

```
weight = 2 if mandatory, otherwise 1
AND coverage = hits / member_count
OR coverage = 1 if any hit, otherwise 0
score = round(100 * sum(weight * coverage)
              / sum(weight), 1)
```

Situation	Exact expected reasoning
Synthetic Mira baseline	Three required groups covered: Python+SQL, Pandas, Linux. Two preferred groups missing: Docker and AWS/GCP. Covered weight 6 / total 8 = 75.0%.
Python/SQL contribution	Their mandatory group is 2/8 of total = 25 points. In its AND group each matched member contributes 12.5 points.
Add hypothetical Docker	The preferred group adds 1/8 = 12.5 points, giving 87.5%. Category self-ratings do not change.
Add hypothetical AWS too	AWS satisfies AWS OR GCP; score 100%. Do not additionally demand GCP or count both alternatives twice.
Move SQL category level to 9	Readiness may change (74.9 -> 79.5 in the synthetic story); named-SQL JD coverage does not change just because its broad rating changed.

Important edge case

A skill toggle changes named JD coverage; a category slider changes company readiness. The two mechanisms are deliberately separate. Fuzzy suggestions/unresolved evidence get review recommendations without promised automatic gains.

GPS must compare one frozen baseline

Request current profile_revision + jd_revision, optional benchmark_hash, skill_overrides=['Docker'] and category_overrides={'SQL':9}. Reject overrides outside the selected JD and stale versions. Baseline/projected results come from the same engines; no profile write occurs.

Never cross this boundary

Do not save a ScenarioResult as a Profile. Recommendations are possibilities, not guarantees, study-time estimates or verified learning. Reset must restore baseline using unchanged actual saved inputs.

06 / INTEGRATION AGREEMENT

Own it. Test it. Publish your part.

Your task is a working module on your own GitHub. Keep the supplied shared contracts and exact output formats so the Scrum Master can integrate the returned code.

Team rule	What you do
Service boundary	matcher.evaluate(profile, jd) -> result dictionary; scenarios.preview(profile, jd, request, bundle) -> preview dictionary. HTTP routes validate the shared result models.
One feature owner	Own skill_matching/backend, frontend, tests and docs. Keep the common support area in your clone. Run your module independently before claiming it works.
One contract source	Read shared/contracts/models.py. OpenAPI and generated TypeScript follow it. Do not maintain a second handwritten schema or change categories alone.
One storage/auth path	Use shared store/document/auth helpers. Do not add a second database, anonymous cloud owner, public CV URL or frontend API secret.
One current record	Use actual returned IDs/revisions, owned by the signed-in user. Pass saved IDs to consumers; never hardcode teaching snapshot IDs.
Your GitHub	Create a private repository on YOUR account. Rename the old origin to scrum-master, set your new origin, review staged files and scan for secrets, then commit and push. See SUBMIT_YOUR_REPOSITORY.md.
Return working evidence	Fill DELIVERY.md with your URL, commit SHA, actual commands/results, three normal and three failure cases, synthetic screenshots and known bugs. If a check fails, fix it or report it honestly; do not copy passing counts.

The official category vocabulary

Every extracted skill has exactly one official code or OTHER. The extra descriptive skill families are filters, not replacement categories. Only the twelve official codes can have numeric ratings or company expectations.

Category	Category
COD - Coding	CLOUD - Cloud
DSA - Data Structures & Algorithms	SQL - SQL
OOD - Object-Oriented Design	SWE - Software Engineering
APTI - Aptitude	SYSD - System Design
COMM - Communication	NETW - Networking
AI - AI	OS - Operating Systems

Keep these meanings separate

Confidence = how clearly the document supports an extraction. Competency = explicit self-rating, null or 1-10. Review = a human checked the output. Verified = independently established; this app's claims remain unverified. Hypothetical = GPS preview only.

07 / RUN AND VERIFY

Your definition of done.

A module must run independently as well as inside the shared app. Record actual commands and failures; a screenshot or a copied passing count is not test evidence.

Use the existing project, not a second app

From the repository root, install the existing requirements/lockfiles if needed. Open two terminals. If the ports are already running, use them; do not kill unknown processes. Never overwrite an existing .env.

```
.\skill_matching\run.ps1 api
# Separate terminal:
.\skill_matching\run.ps1 web
# Real standalone API check:
.\skill_matching\run.ps1 check
```

Clone the full repo and install root dependencies (README). Run these from its root. Open http://127.0.0.1:5173/; Load synthetic inputs supplies upstream fixtures. No key is needed for rules. A Groq key belongs only in the private root .env; restart Python after editing it.

Test	Pass means
Two candidates x two JDs	Explainably different scores; deterministic repeats and bounded values.
AND/OR, duplicates, aliases	Correct group arithmetic; Java/JavaScript not equivalent; no double counting.
Unknown/conflicting/unreviewed evidence	Unresolved/provisional output, no automatic false match.
Empty JD / no company benchmarks	Empty is unscorable; JD matching still works without company data.
Scenario invalid/stale input	Reject skill outside JD, invalid category value and stale revision/hash.
Preview/reset/non-mutation	Expected deltas and identical saved profile JSON before/after; same engines used.

Your focused commands

```
.\.venv\Scripts\python.exe -m pytest skill_matching/tests -q
npm run test -- skill_matching/frontend
```

Before returning: run typecheck, lint, build and the standalone browser check. Use the project virtualenv for Python. Keep assertions meaningful and record actual results.

Error behavior to demonstrate

422 = invalid/unreadable input; 413 = size limit; 415 = unsupported file; 404 = missing/unowned record; 409 = stale revision; 503 = unavailable storage/benchmark. Groq may fall back and return a successful Extraction with llm_status=unavailable instead of an HTTP error. Show the status, not just HTTP 200.

08 / LIVE DEMO AND PRESENTATION

Show it. Explain it. Hand it over.

Suggested speaking time: four minutes per module. Five modules use about 20 minutes, leaving 10 for the problem, architecture, challenge and close within the 30-minute presentation. Q&A is separate (15 minutes).

Reproducible synthetic demo

1. Select the reviewed synthetic data JD and saved Mira profile. Click Analyze this match: baseline 75.0%; four matched decisions and three missing names across two groups.

2. Open Python in Matched: show JD quote, candidate quote, 12.50 points and next action. Open Docker gap: no candidate evidence, 0.00 points.

3. Choose Explore in Career GPS. This navigation must not automatically apply the missing skill. Toggle Docker: projected match 87.5%, readiness stays 74.9%.

4. Set SQL hypothetical level to 9: readiness 79.5%; named match remains 87.5%. Add AWS: match 100% because AWS OR GCP is satisfied.

5. Reset scenario: 75.0% / 74.9%. Return to Profile Builder: SQL still 4; no actual Docker/AWS claim was added.

What to say and show

Time	Speaking cue
0:00-0:35	My module compares this candidate with this particular JD. Company readiness is a separate result.
0:35-1:25	Show 75% and its missing list. Explain weight 6 of 8 rather than a mysterious AI-generated number.
1:25-2:15	Open Python and Docker reasoning chains. Distinguish supported evidence from an actual capability gap.
2:15-3:20	Simulate Docker and SQL. Show two separate score changes and explain why they differ.
3:20-4:00	Reset and verify saved inputs unchanged. Close with an honest limitation and the complete five-module journey.

Transition to the next speaker

Together the five modules connect original job text, candidate claims, saved profile, company readiness and reversible next-step exploration.

Use this language honestly

Say which parts you implemented, adapted, tested or learned. If showing the working team baseline, call it the integrated team implementation. Do not claim a live model, official benchmark or feature history that has not actually been demonstrated.

09 / FIELD REFERENCE AND QUESTIONS

Keep this beside your editor.

These are exact field names from the current Python contracts. Defaults fill omitted optional fields; partial excerpts are not safe replacements for full PATCH payloads.

Group	Exact fields / meaning
Matching identity	jd_source_file, jd_id, jd_revision, profile_id, profile_revision
Matching explanation	match_score, matched_skills, missing_skills, ambiguous_matches, contributions, rules_version, status, methodology, receipt_id
Decision record	requirement_id, skill_name, category_code, matched, requirement_type, jd_evidence, candidate_evidence, decision_type, reason, suggested_review, recommended_action, score_contribution
Contribution group	requirement_id, operator, requirement_type, skills, weight, coverage, weighted_contribution, requires_review, member_count, matched_count, score_contribution
Scenario request	profile_id, jd_id, profile_revision, jd_revision, optional company_id/benchmark_mode/benchmark_hash; skill_overrides; category_overrides
Scenario result	hypothetical=true, baseline_match, projected_match, baseline_readiness, projected_readiness, match_delta, readiness_delta, recommendations, profile_revision, notice

Questions you should answer without memorizing code

Why not match only by category? Knowing one AI technology does not prove another named tool. Exact names/reviewed aliases are safer; fuzzy similarity only suggests review.

Can 100% mean guaranteed selection? No. It means coverage of the reviewed extracted requirement groups, not a hiring probability or verified competence.

Does the simulation improve the real profile? No. It copies inputs and recalculates. Real changes require explicit editing/saving in Profile Builder.

Source of truth and current limits

Read your module's README.md, PLAN.md, CONTRACT.md and USAGE.md; shared/contracts/models.py; docs/BUILD_PLAN.md; docs/DEPLOYMENT_SETUP.md; docs/LLM_EXTRACTION_IMPLEMENTATION.md; verification/RESULTS.md; and docs/TEAM_PRESENTATION_CHECKLIST.md. Full current validated synthetic JSON snapshots are under data/synthetic/handoff/. Snapshot IDs are not live IDs.

Original Participant Brief: pp.7-10 (roles/contracts), p.11 (official test pack), p.13 (stack/done). Event Plan p.4: 30 minutes presentation + 15 Q&A. Original baseline suite (see new checkout results in verification/RESULTS.md): 134 backend, 17 frontend and 14 browser tests. Live Groq quality, cloud RLS/deployment and official organizer-pack/benchmark acceptance remain pending. All guide examples are synthetic.